SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 1

Analytical Problem Solving

Name of the Student	G Venkata Ganesh
Reg. No	192425381

COURSE NAME: Deep Learning **COURSE CODE: MLA0403**
FACULTY : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

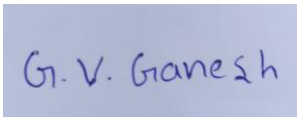
Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I G Venkata Ganesh / 192425381 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

G VENKATA GANESH

Name of the student:

QUESTIONS:05 **Total Marks:25**

Analytical Problem Solving

1. Learning Algorithms (Optimization Behavior)

A machine learning model is trained using gradient descent on a convex loss function. Initially, the learning rate is set very high, and later it is gradually decreased.

Question:
Analyze how the choice of a high initial learning rate followed by decay affects convergence. Under what conditions can this strategy outperform a constant learning rate? Discuss possible risks and justify with reasoning.

2. Maximum Likelihood Estimation (MLE)

Suppose you are given a dataset generated from a Gaussian distribution with unknown mean μ and known variance σ^2

Question:
Derive the Maximum Likelihood Estimator for μ, and analytically explain how the estimate changes when:

- The dataset contains outliers
- The sample size increases
  What does this imply about the robustness of MLE?

3. Building a Machine Learning Algorithm (Model Selection)

You are designing a classification algorithm for a dataset with 10,000 features but only 500 training samples.

Question:
Analyze the challenges involved in training such a model. Propose a strategy (algorithm pipeline) to handle this situation and justify each step (e.g., feature selection, regularization, dimensionality reduction). Explain how your approach mitigates overfitting.

4. Neural Networks (Multilayer Perceptron - MLP)

Consider a Multilayer Perceptron with one hidden layer using sigmoid activation functions.

Question:
Explain analytically why adding more hidden neurons increases the representational power of the network. Then, reason why this does not always lead to better performance. Support your answer using concepts like overfitting and generalization.

5. Backpropagation, SGD & Curse of Dimensionality

A deep neural network is trained using Stochastic Gradient Descent (SGD) on a very high-dimensional dataset.

Question:
Analyze how the **curse of dimensionality** impacts:

- Gradient updates in backpropagation
- Convergence speed of SGD
- Model generalization

Propose at least two techniques to overcome these issues and justify them analytically.

ANSWERS

1.Learning Algorithms (Optimization Behavior)

Effect of a High Initial Learning Rate

- A high learning rate allows the model to take large optimization steps during the early stages of training.
- This enables the algorithm to reach the neighbourhood of the global minimum much faster than a small learning rate.
- For a convex loss function, the optimizer moves quickly toward the optimum if the learning rate is not excessively large.

Effect of Learning Rate Decay

- As training progresses, gradually reducing the learning rate results in smaller and more precise parameter updates.
- This helps stabilize convergence and prevents oscillations around the optimal solution.
- The model can fine-tune the parameters with higher accuracy.

When This Strategy Outperforms a Constant Learning Rate

- The optimization problem has a smooth convex loss function.
- The initial parameters are far from the optimum.
- Faster convergence is desired without sacrificing final accuracy.
- The learning rate decreases gradually to allow precise convergence near the minimum.

Possible Risks

- If the initial learning rate is too high, the optimizer may overshoot the minimum.
- Training may oscillate or even diverge instead of converging.
- Very rapid learning-rate decay can slow learning before reaching the optimum.
- Very slow decay may continue causing oscillations.

Justification

- Early stage: Fast exploration of the parameter space.
- Later stage: Stable convergence with improved accuracy.

Thus, this strategy often converges faster and achieves a lower final loss than using a constant learning rate.

2. Maximum Likelihood Estimation (MLE)

For observations

$$x_1, x_2, \ldots, x_n \sim N(\mu, \sigma^2)$$

where σ^2 is known and μ is unknown.
The likelihood function is

$$L(\mu) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(\quad -\frac{(x_i - \mu)^2}{2\sigma^2} \right)$$

Taking the logarithm,

$$\log L(\mu) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(x_i - \mu)^2$$

Differentiate with respect to μ:

$$\frac{d}{d\mu}\log L(\mu) = \frac{1}{\sigma^2}\sum_{i=1}^{n}(x_i - \mu)$$

Setting the derivative equal to zero,

$$\sum_{i=1}^{n}(x_i - \mu) = 0$$

Therefore,

$$\boxed{\hat{\mu} = \frac{1}{n}\sum_{i=1}^{n} x_i}$$

Hence, the MLE of μ is the sample mean.

Effect of Outliers

- The sample mean is highly sensitive to extreme values.
- Even one large outlier can significantly shift the estimated mean.

Effect of Increasing Sample Size

- As the number of samples increases, the estimate becomes more stable.
- Variance of the estimator decreases as

$$Var(\hat{\mu}) = \frac{\sigma^2}{n}$$

Implication for Robustness

- Efficient under the Gaussian assumption.
- Consistent for large datasets.

3. Building a Machine Learning Algorithm (Model Selection)

Challenges

The dataset contains:

- 10,000 features
- 500 training samples

This creates a high-dimensional, low-sample-size problem, leading to:

- Overfitting
- High computational cost
- Redundant and irrelevant features
- Poor generalization

Proposed Algorithm Pipeline

Step 1: Data Preprocessing

- Handle missing values.
- Normalize or standardize features.

Justification: Ensures all features are on the same scale.

Step 2: Feature Selection

Use methods such as:

- Mutual Information
- Chi-Square
- Recursive Feature Elimination (RFE)

Reduce features from 10,000 to a few hundred.

Justification: Removes irrelevant features and reduces model complexity.

Step 3: Dimensionality Reduction

Apply PCA.

Justification

- Removes feature redundancy.
- Preserves maximum variance.
- Reduces computational cost.

Step 4: Regularization

Use:

- L1 (Lasso)
- L2 (Ridge)

Justification

Regularization penalizes large weights and prevents overly complex models.

Step 5: Classification

Use classifiers such as:

- Support Vector Machine (SVM)
- Logistic Regression

These perform well with high-dimensional datasets.

Step 6: Cross-Validation

Use k-fold cross-validation.

Justification

Provides reliable estimation of model performance and reduces overfitting.

How This Mitigates Overfitting

- Feature selection removes noisy variables.
- PCA reduces dimensionality.
- Regularization limits model complexity.
- Cross-validation improves generalization.

Thus, the model becomes more accurate and performs better on unseen data.

4. Neural Networks (Multilayer Perceptron - MLP)

A Multilayer Perceptron consists of an input layer, one hidden layer, and an output layer.

Why More Hidden Neurons Increase Representational Power

Each hidden neuron learns a different nonlinear feature.

Adding more neurons allows the network to:

- Learn more complex decision boundaries.
- Capture nonlinear relationships.
- Approximate increasingly complex functions.

According to the Universal Approximation Theorem, an MLP with enough hidden neurons can approximate any continuous function.

Why More Neurons Do Not Always Improve Performance

Although representational power increases, excessive neurons can cause:

Overfitting

- The model memorizes training data instead of learning general patterns.
- Training accuracy becomes very high.
- Testing accuracy decreases.

Higher Computational Cost

- More parameters increase training time.
- Memory usage also increases.

Poor Generalization

An overly complex model performs well on training data but poorly on unseen data.

Generalization

A well-generalized model captures underlying data patterns instead of noise.

Methods to improve generalization include:

- Regularization
- Dropout
- Early stopping
- Cross-validation

Conclusion

Increasing hidden neurons improves learning capacity, but beyond an optimal point it increases overfitting and reduces generalization. Therefore, selecting an appropriate network size provides the best balance between model complexity and predictive performance.

5. Backpropagation, SGD & Curse of Dimensionality

The **curse of dimensionality** refers to problems that arise when the number of features becomes extremely large.

Impact on Gradient Updates in Backpropagation

- The network must compute gradients for a very large number of parameters.
- Many features contribute very little useful information.
- Gradients become noisy and less informative.
- Optimization becomes more difficult due to sparse data in high-dimensional space.

Impact on Convergence Speed of SGD

- SGD updates only one or a few samples at a time.
- In high-dimensional datasets, gradient estimates have higher variance.
- More iterations are required to reach convergence.
- Training becomes slower and less stable.

Impact on Model Generalization

- High-dimensional models have many parameters.
- With limited training samples, the model easily memorizes the training data.
- Test accuracy decreases because the model fails to generalize.
- The risk of overfitting increases significantly.

Methods to Reduce the Curse of Dimensionality

- Feature selection to remove irrelevant variables.
- PCA or other dimensionality reduction techniques.
- Regularization (L1/L2).
- Dropout in neural networks.
- Increasing the amount of training data.
- Early stopping during training.

Conclusion

The curse of dimensionality makes optimization slower, increases gradient noise, and reduces generalization. Applying dimensionality reduction, regularization, and feature selection improves convergence, reduces overfitting, and enhances model performance.